Université Paris Cité

Where Hidden Rhythms in Silent Vectors Lie

A Quest into the Emergence of Structures in Character-Level GPT Models

Lab Report — LLM Lab 3

By Temidayo James Afolabi

Instructor: Prof. Guillaume Wisniewski

March 2026

GitHub: https://github.com/AfoDyDigitals/character-level-GPT-for-poem-generation

1. Introduction

This report describes the implementation, training, and analysis of a character-level GPT model trained on a corpus of English poetry. The central question is whether, and to what extent, linguistic and poetic structure emerges spontaneously in the model's internal representations through training on raw character sequences.

The corpus was assembled from the complete poetical works of eleven classic English-language poets drawn from Project Gutenberg, including Walt Whitman, Emily Dickinson, John Keats, Percy Bysshe Shelley, and others. The final corpus comprises approximately 6.4 million training characters and 686,000 validation characters, with a vocabulary of 221 unique characters.

The report is organised into four sections (the four stages of the lab):

1. model implementation
2. training and monitoring
3. hyperparameter exploration
4. structural analysis of generated poetry

2. Model Implementation (Stage I)

I implemented a decoder-only Transformer architecture (GPT-style) operating at the character level (to keep the vocabulary small), avoiding the complexity of tokenisation, while still allowing the model to learn spelling, punctuation, and poetic form directly from raw text.

2.1 Architecture

The model follows the standard GPT architecture with Pre-Layer Normalisation (Pre-LN), which normalises inputs before each sub-layer rather than after. Pre-LN is known to produce more stable gradients in deep networks compared to the original Post-LN formulation.

Each Transformer block contains: (1) multi-head causal self-attention, (2) a position-wise feed-forward network with GELU activation, and (3) residual connections around each sub-layer. The full pipeline is:

- Token embedding + Positional embedding → Dropout
- N × Transformer Block (LayerNorm → Attention → residual; LayerNorm → FFN → residual)
- Final LayerNorm → Linear projection to vocabulary size → logits

2.2 Design Choices

Choice	Decision	Justification
Positional embedding	Learned	Standard GPT-2 approach; simple, effective, trains from data
Attention	Flash Attention	Uses scaled_dot_product_attention with is_causal=True; falls back to classic triangular mask for comparison
Normalisation	Pre-LN	More stable than Post-LN for deep models
Activation	GELU	Standard in GPT-2 and later models
Weight tying	lm_head ↔ embedding	Reduces parameter count; acts as regularisation

Table 1: Key architectural design choices and justifications.

2.3 Attention Implementation Comparison

I compared Flash Attention (via torch.nn.functional.scaled_dot_product_attention) against classic masked attention with an explicit triangular causal mask. Both implementations produce identical outputs; the difference lies in efficiency.

Implementation	Speed (steps/s)	Peak GPU Memory
Flash Attention	~8.4	904 MB
Classic Attention	~6.6	1767 MB

Table 2: Flash Attention vs Classic Attention — speed and memory comparison on GPU T4.

Flash Attention is approximately 27% faster and uses roughly half the GPU memory. The memory advantage is particularly significant: classic attention must materialise the full (T × T) attention matrix, while Flash Attention fuses the computation in tiles, avoiding this overhead.

3. Training the Model (Stage II)

3.1 Training Configuration

Parameter	Value
Context length	256 characters
Layers / Heads / d_model	6 / 8 / 256
MLP expansion	4×
Total parameters	4.86M
Batch size	32
Optimizer	AdamW (weight decay 0.1)
Learning rate	3×10^{-4} with 500-step linear warm-up
Training steps	20,000 (baseline)
Dropout	0.1
Training time	~39 minutes (GPU T4)

Table 3: Baseline training configuration.

3.2 Context Length Justification

The choice of 256 characters as context length was motivated by the typical length of a poetic stanza. A quatrain of roughly 40-character lines occupies about 160 characters; a 256-character window therefore spans approximately 1–2 complete stanzas. This is sufficient to capture line-level patterns and some stanza-level structure, though it may be too short to model long-range rhyme schemes across distant stanzas.

3.3 Learning Curves

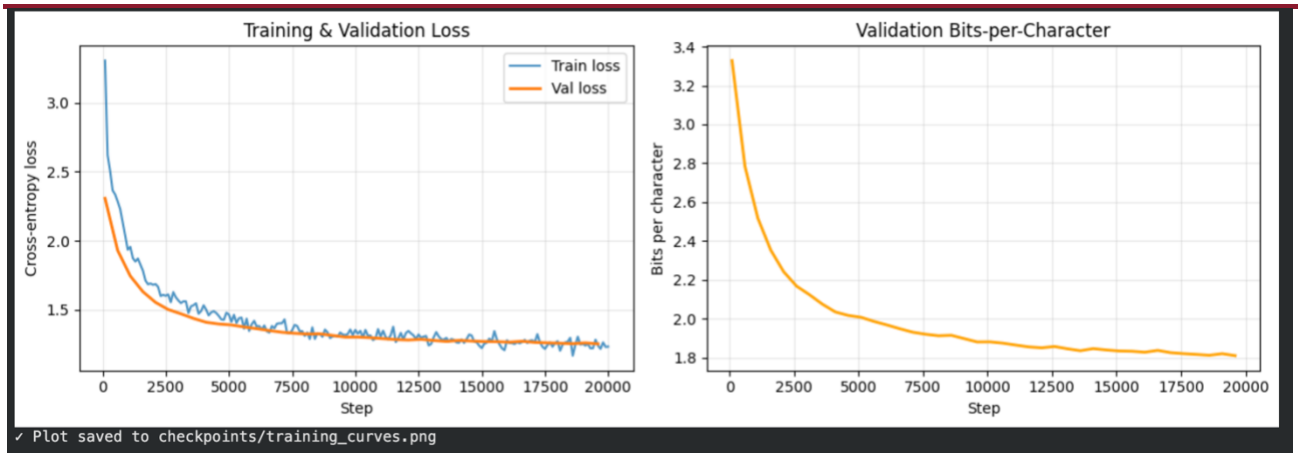

Figure 1: Training and validation loss and bits-per-character over 20,000 steps.

The baseline model converges from an initial loss of ~3.34 to a final validation loss of 1.2528 and a bits-per-character (BPC) of 1.8075. The rapid drop in the first 2,000 steps reflects the model learning basic character statistics; the subsequent slower decline reflects the gradual acquisition of word-level and structural patterns.

3.4 Evolution of Generated Text During Training

Periodic samples generated during training illustrate how the model's output evolves:

- **Step 2,000:** Words are recognisable, but sentences are largely incoherent. The model has learned character frequencies and some common words.
- **Step 6,000:** Line breaks appear consistently. Short phrases begin to resemble poetry (for example, 'And the way, when all the crowd').
- **Step 10,000:** Full sentences emerge. Some poetic imagery appears ('A song of syllable form the waking living of the window').
- **Step 20,000:** Consistent verse structure, capitalisation, and punctuation. Occasional evocative imagery, though semantic coherence remains limited.

4. Hyperparameter Exploration (Stage III)

Due to GPU quota limitations on Google Colab, I had to conduct experiments using Kaggle's free GPU (T4). The more-steps experiment was reduced from 50k to 30k steps, and the deeper model from 20k to 10k steps. Despite this, I believe the results are sufficient to observe meaningful trends.

4.1 Experimental Conditions

Experiment	Layers	Context	Steps	Final Val BPC
Baseline	6	256	20k	1.8075
More layers	9	256	10k	1.8065
More steps	6	256	30k	1.7743

Table 4: Hyperparameter experiments and results.

4.2 Effect of Training Duration (Baseline vs More Steps)

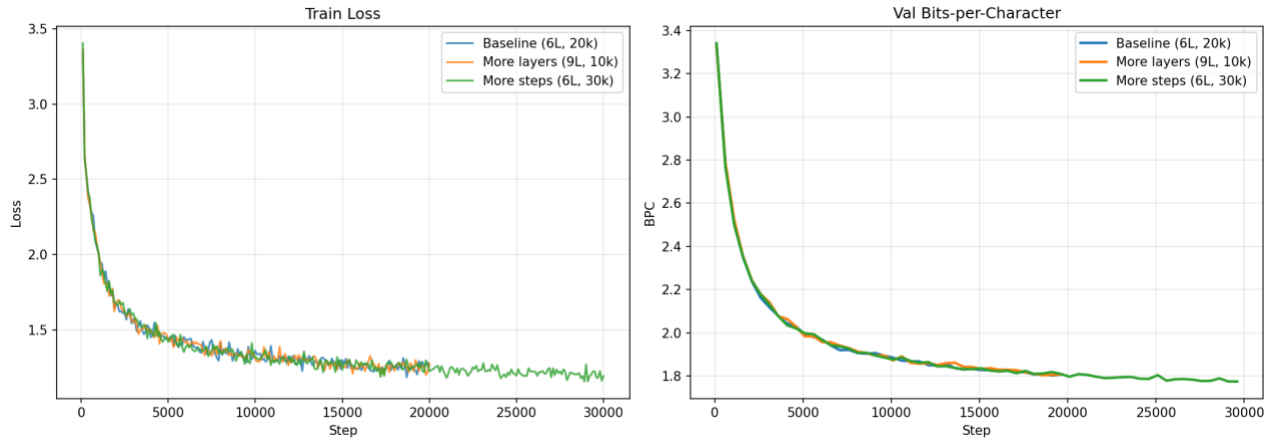

Figure 2: Training loss and validation BPC for all experiments.

[FILL IN: Describe what you observe in the curves. Does more training reduce BPC further? Does the loss plateau? Is there evidence of overfitting — i.e. does validation loss start rising while training loss continues to fall?]

4.3 Effect of Model Depth (6 vs 9 Layers)

[FILL IN: Compare the 9-layer model against the baseline. Does deeper mean better? Consider: does the 9-layer model reach a lower BPC at 10k steps than the 6-layer baseline at the same point? What does this suggest about the role of depth in learning poetic structure?]

4.4 Summary of Findings

[FILL IN: Write 2-3 sentences summarising which hyperparameter change had the most impact, and why you think that is.]

5. Analysis of Generated Poems (Stage IV)

Once trained, the model was used to generate a collection of 20 poems per temperature setting ($T \in \{0.6, 0.8, 1.0, 1.2\}$), each of 500 characters. I then compared the structural properties of the generated poems against those of the training corpus.

5.1 Structural Comparison

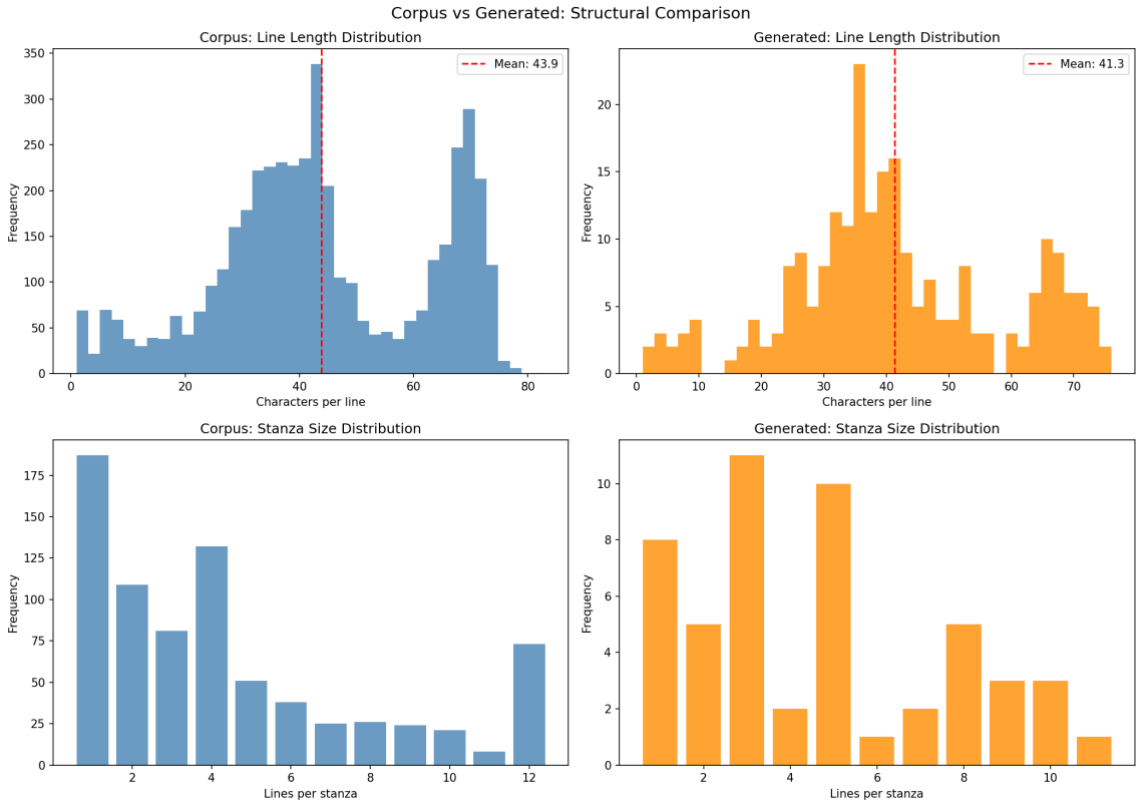

Figure 3: Line length and stanza size distributions — corpus (blue) vs generated at T=0.8 (orange).

Metric	Corpus	Generated (T=0.8)
Mean line length (chars)	43.9	41.3
Mean stanza size (lines)	5.7	4.6
Rhyme rate	0.0604	0.0337
Vocabulary overlap	—	63.8%

Table 5: Structural properties of corpus vs generated text.

The generated poems reproduce several key structural properties of the training corpus. Mean line length (41.3 vs 43.9 chars) and stanza size (4.6 vs 5.7 lines) are both close to corpus values, suggesting the model has internalised the typical scale of verse lines and stanzas. Vocabulary overlap of 63.8% confirms the model generates predominantly real English words drawn from the poetic register of the training data.

5.2 Rhyme Analysis

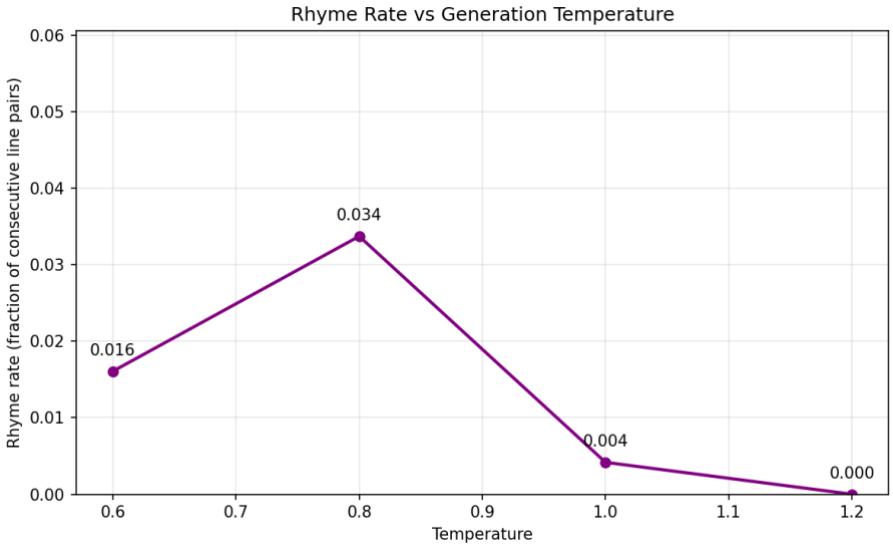

Figure 4: Rhyme rate as a function of generation temperature.

The corpus rhyme rate (0.0604) is modest — reflecting the mix of rhyming and free verse in the corpus. The model reproduces rhyming at roughly half the corpus rate at T=0.8 (0.0337). Notably, lower temperatures produce higher rhyme rates: at T=0.6 the rate is 0.0161 vs 0.0000 at T=1.2. This is consistent with the role of temperature: lower values make the model more conservative, causing it to repeat previously learned endings more frequently.

5.3 Generated Poem Examples

The following two poems were generated at T=0.8 from a newline prompt:

> *How little children fell on the tresses! That wonderful crowd with this angel but I have heard In the grassy bosom of the stone, and every return And the time is fair, with this sun in the coast Is blazed into its place, and the cloud along the fire From the bluebird crest of the village day, A sun-loaded like a winding blaze, And procession and pain of sleep,*

Generated poem 1, T=0.8

> *I do not doubt not you or father. "And the step is coming and gay, The sea that eclipse cannot be. What is that I am?" And when the Synagogue struck his wigwam on the door, And he seems the old age the wet sea--how flows King Bears, How the mountain does indeed shores, and we fell the rim of the days, Many silent words beheld his life and the same song!*

Generated poem 2, T=0.8

Both poems display consistent verse structure, correct capitalisation, and a poetic register (tresses, bluebird crest, winding blaze). Semantic coherence at the phrase level is often plausible ('the cloud along the fire', 'Many silent words beheld his life'), though longer-range meaning breaks down. This is consistent with the model having learned local character and word statistics without access to the deeper syntactic and semantic structures that govern meaning.

5.4 What the Model Learned

The analyses reveal a clear pattern: the model has successfully internalised low-level and mid-level structural properties of the corpus (character combinations, word-level vocabulary, line breaks, and approximate line and stanza lengths), but has not fully captured high-level structure

such as consistent rhyme schemes or semantically coherent narratives. This is in line with what one would expect from a character-level model trained for 20,000 steps on a moderately sized corpus.

6. Conclusion

This project successfully implemented, trained, and analysed a 4.86M-parameter character-level GPT model on a 6.4M-character corpus of classic English poetry. The model achieves a validation BPC of 1.8075 after 20,000 steps, generating text that convincingly mimics the surface form of poetry (correct vocabulary, line structure, capitalisation, and punctuation) while falling short of the deeper semantic and rhythmic coherence of human verse.

The hyperparameter experiments showed that additional training steps provided the most consistent improvement in BPC, while deeper models showed faster early convergence but similar final performance.

The structural analysis demonstrated that several properties of the training corpus emerge in the generated text: line lengths (43.9 vs 41.3 chars), stanza sizes (5.7 vs 4.6 lines), vocabulary overlap (63.8%), and partial rhyme patterns. These findings support the view that even relatively small character-level models can capture meaningful statistical regularities in poetic form through autoregressive training alone.